CIVICFIX AI

AI-Assisted Smart City Complaint & Infrastructure Management App

Advanced BTech CSE Final-Year Project

React Native / React • Spring Boot • Python AI Service • PostgreSQL • Maps • Docker

Project concept, requirements, architecture, database schema, API plan, starter code, AI design, testing, deployment and GitHub workflow.

This is a project implementation guide and starter reference. Build and test the remaining modules, use synthetic demo data, and follow your college's academic-integrity rules.

1. Project abstract

CivicFix AI is a civic issue reporting and resolution platform. Citizens submit reports about public infrastructure problems with a description, category, photo and map location. The system routes reports to the relevant municipal department, lets field staff update progress, and gives administrators a dashboard for workload and resolution analytics.

Problem being addressed

Civic complaints are often submitted through disconnected channels. Citizens may not know where a complaint stands, duplicate reports can be difficult to consolidate, and administrators may lack a live view of unresolved issues. CivicFix AI creates one traceable workflow from reporting to closure.

Project goals

• Make civic issue reporting simple and accessible.

• Give each complaint a reference number and visible status history.

• Help administrators assign complaints to the right department and staff.

• Use AI-assisted image/category suggestions as decision support, not as an automatic final judgement.

• Show maps and analytics for operational planning.

Important scope: This is an academic prototype. Do not claim integration with a real municipality unless you have written authorization and a working integration.

2. Roles and features

Role	Core features
Citizen	Sign up/login, report issue, attach photo, select map location, track status, add comments and feedback.
Field Staff	View assigned tasks, accept work, add progress notes, upload completion evidence, request review.
Department Officer	View department queue, assign staff, set priority, merge duplicate reports, approve closure.
System Admin	Manage users/departments/categories, monitor reports, view analytics, review audit events.

Issue categories

• Road and pothole damage

• Streetlight outage

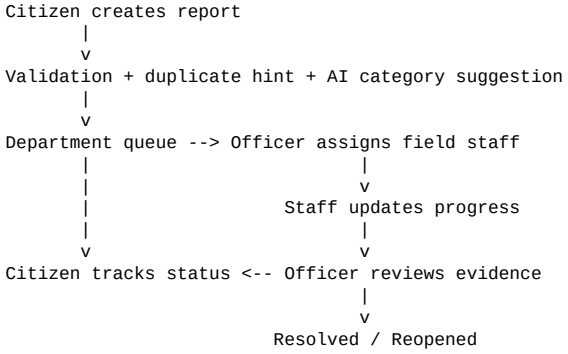
• Garbage collection / overflowing bins

• Water leakage

• Drainage and sewer issues

• Public property damage

Workflow

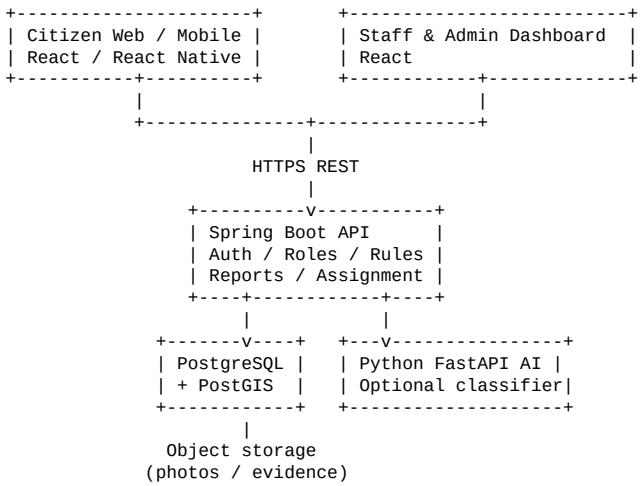


3. Recommended technology stack

Layer	Technology	Purpose
Mobile/web UI	React Native (Expo) or React.js	Citizen app and admin/staff portal
Backend API	Java 21 + Spring Boot	Authentication, workflows, business rules
AI microservice	Python + FastAPI	Optional image/category suggestion service
Database	PostgreSQL + PostGIS	Relational data and geospatial queries
File storage	Local dev storage; object storage in deployment	Complaint images and evidence
Maps	Leaflet / OpenStreetMap or approved map provider	Location and issue map
Testing	JUnit, Mockito, Pytest, Postman	Unit, integration and API testing
Deployment	Docker Compose; cloud hosting	Reproducible deployment

For a simpler college build, use React.js for both citizen and admin experiences. Add React Native only if a mobile app is a required deliverable.

4. System architecture



Suggested repository layout

```
civicfix-ai/
  backend/
    src/main/java/com/civicfix/
      config/ controller/ dto/ entity/
      repository/ security/ service/
    src/main/resources/
  ai-service/
    app/main.py
    requirements.txt
  frontend/
    src/api/ components/ pages/ hooks/
  docs/
    architecture/ screenshots/ API.md
  docker-compose.yml
  README.md
```

5. Database design

Table	Key columns
users	id PK, full_name, email UNIQUE, password_hash, role, enabled, created_at
departments	id PK, name UNIQUE, description
staff_profiles	id PK, user_id FK UNIQUE, department_id FK

Table	Key columns
reports	id PK, reference_code UNIQUE, citizen_id FK, category_id FK, description, latitude, longitude, address, priority, status, create
categories	id PK, name UNIQUE, department_id FK
report_media	id PK, report_id FK, storage_key, media_type, uploaded_at
assignments	id PK, report_id FK, staff_id FK, assigned_by FK, assigned_at, active
status_history	id PK, report_id FK, changed_by FK, old_status, new_status, note, changed_at
comments	id PK, report_id FK, user_id FK, body, created_at
feedback	id PK, report_id FK, rating, comment, created_at

Relationship summary

• One citizen can create many reports.

• Each report belongs to one category and may be assigned to a department through that category.

• A report can have multiple assignment and status-history records.

• One staff member can receive multiple assignments.

• A report can contain multiple media attachments and comments.

PostgreSQL starter schema

```
CREATE TABLE users (
  id BIGSERIAL PRIMARY KEY,
  full_name VARCHAR(120) NOT NULL,
  email VARCHAR(180) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  role VARCHAR(24) NOT NULL CHECK
    (role IN ('CITIZEN','STAFF','OFFICER','ADMIN')),
  enabled BOOLEAN NOT NULL DEFAULT TRUE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE departments (
  id BIGSERIAL PRIMARY KEY,
  name VARCHAR(100) UNIQUE NOT NULL
);

CREATE TABLE categories (
  id BIGSERIAL PRIMARY KEY,
  name VARCHAR(100) UNIQUE NOT NULL,
  department_id BIGINT REFERENCES departments(id)
);

CREATE TABLE reports (
  id BIGSERIAL PRIMARY KEY,
  reference_code VARCHAR(30) UNIQUE NOT NULL,
  citizen_id BIGINT NOT NULL REFERENCES users(id),
  category_id BIGINT NOT NULL REFERENCES categories(id),
  description TEXT NOT NULL,
  latitude DOUBLE PRECISION,
  longitude DOUBLE PRECISION,
  address VARCHAR(300),
  priority VARCHAR(12) DEFAULT 'NORMAL',
  status VARCHAR(20) NOT NULL DEFAULT 'SUBMITTED',
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_reports_status_created
  ON reports(status, created_at DESC);
CREATE INDEX idx_reports_category
  ON reports(category_id);
```

For production geospatial filtering, enable PostGIS and store coordinates as a geography/geometry column with a spatial index. Validate latitude/longitude ranges and never trust coordinates without server-side validation.

6. Backend API design

Method / endpoint	Access	Purpose
POST /api/auth/register	Public	Create citizen account
POST /api/auth/login	Public	Authenticate and issue access token
POST /api/reports	Citizen	Submit a new report
GET /api/reports/mine	Citizen	List own reports
GET /api/reports/{id}	Owner / assigned staff / officer	Report details subject to access policy
GET /api/officer/reports	Officer	Department queue with filters
POST /api/reports/{id}/assign	Officer	Assign staff member
PATCH /api/reports/{id}/status	Assigned staff / officer	Update workflow status
POST /api/reports/{id}/comments	Authorized user	Add report comment
GET /api/admin/analytics	Admin	Summary metrics and trends

Example report request

```
POST /api/reports
Authorization: Bearer <access-token>
Content-Type: application/json

{
  "categoryId": 2,
  "description": "Streetlight is not working near the park entrance.",
  "latitude": 26.9124,
  "longitude": 75.7873,
  "address": "Demo location, Jaipur"
}
```

Example response

```
{
  "referenceCode": "CFX-2026-000124",
  "status": "SUBMITTED",
  "message": "Your report has been received."
}
```

Status model

Suggested statuses: SUBMITTED → UNDER_REVIEW → ASSIGNED → IN_PROGRESS → RESOLVED. Additional outcomes can include REJECTED (with reason), DUPLICATE (linked to a canonical report) and REOPENED. Enforce allowed transitions in the service layer and record every change in status_history.

7. Backend starter code

Spring Boot report DTO

```
package com.civicfix.dto;

import jakarta.validation.constraints.*;

public record CreateReportRequest(
    @NotNull Long categoryId,
    @NotBlank @Size(max = 3000) String description,
    @DecimalMin("-90.0") @DecimalMax("90.0") Double latitude,
    @DecimalMin("-180.0") @DecimalMax("180.0") Double longitude,
    @Size(max = 300) String address
) {}
```

Controller pattern

```
@RestController
@RequestMapping("/api/reports")
public class ReportController {
    private final ReportService reportService;

    public ReportController(ReportService reportService) {
        this.reportService = reportService;
    }

    @PostMapping
```

```
    @PreAuthorize("hasRole('CITIZEN')")
    public ResponseEntity<ReportResponse> create(
        @Valid @RequestBody CreateReportRequest request,
        Authentication authentication
    ) {
        ReportResponse created =
            reportService.createForCitizen(authentication.getName(), request);
        return ResponseEntity.status(HttpStatus.CREATED).body(created);
    }
}
```

Implement ReportService to verify the category exists, generate a non-guessable public reference code, persist the report and initial status-history entry in one transaction. Resolve the citizen from the authenticated principal; do not accept citizenId from the request body.

Repository pattern

```
public interface ReportRepository
        extends JpaRepository<Report, Long> {
    Page<Report> findByCitizenEmailOrderByCreatedAtDesc(
        String email, Pageable pageable
    );
    Page<Report> findByStatusOrderByCreatedAtDesc(
        String status, Pageable pageable
    );
}
```

The repository signatures assume matching entity relationships and property paths; adjust them to your actual entity mapping and compile/test them as you implement the model.

8. AI component – responsible use

AI is an optional decision-support feature. A first version can suggest a category from a user-provided image or text description. A human officer remains responsible for confirming the category, priority and assignment.

Possible AI pipeline

• Citizen submits text and optional image.

• Validate file type, size and image content; strip unnecessary metadata.

• Send an authorized, minimal payload to the AI service.

• Return a category suggestion and confidence score.

• Show the suggestion to the citizen or officer for confirmation.

• Store model version and suggestion outcome for evaluation.

FastAPI starter endpoint

```
from fastapi import FastAPI
from pydantic import BaseModel, Field

app = FastAPI(title="CivicFix AI Service")

class SuggestionRequest(BaseModel):
    description: str = Field(min_length=5, max_length=3000)

@app.post("/suggest-category")
def suggest_category(payload: SuggestionRequest):
    text = payload.description.lower()

    # Baseline rules only; replace with a trained and evaluated model.
    rules = {
        "ROAD_POTHOLE": ["pothole", "road damage", "crater"],
        "STREETLIGHT": ["streetlight", "lamp", "light not working"],
        "GARBAGE": ["garbage", "waste", "overflowing bin"],
        "WATER_LEAK": ["water leak", "pipe burst", "leakage"]
    }
    for category, terms in rules.items():
        if any(term in text for term in terms):
            return {
                "suggestedCategory": category,
                "confidence": 0.55,
                "model": "keyword-baseline-v1"
            }

    return {
        "suggestedCategory": "OTHER",
```

```
        "confidence": 0.0,
        "model": "keyword-baseline-v1"
    }
```

This is a transparent keyword baseline, not machine learning. For an ML extension, curate a permissioned, labelled dataset, split it into train/validation/test sets, report per-class precision/recall and a confusion matrix, and evaluate errors before any real-world use.

9. Frontend screens

Screen	Main UI elements
Welcome / Login	Project intro, sign-in, register, help and privacy note.
Citizen home	Create report action, recent reports, status summary.
New report	Category, description, photo picker, map pin, address and submit.
Report tracking	Reference code, timeline, comments and resolution evidence.
Staff task list	Assigned reports, location, priority, status update and notes.
Officer dashboard	Department queue, filters, assignment controls and duplicate review.
Admin analytics	Reports by category/status, resolution time trend and workload.

Example React report card

```
function ReportCard({ report }) {
  return (
    <article className="report-card">
      <div className="row">
        <strong>{report.referenceCode}</strong>
        <span>{report.status}</span>
      </div>
      <h3>{report.categoryName}</h3>
      <p>{report.description}</p>
      <small>Submitted: {new Date(report.createdAt).toLocaleString()}</small>
    </article>
  );
}
```

Basic styling

```
.report-card {
  background: #fff;
  border: 1px solid #dbe5ed;
  border-radius: 12px;
  padding: 16px;
  margin: 12px 0;
}
.row {
  display: flex;
  justify-content: space-between;
  gap: 12px;
}
```

10. Local development setup

On macOS, install Java 21, Node.js LTS, Python 3.11+, PostgreSQL and Git. You can use Homebrew if installed.

```
java -version
node -v
python3 --version
psql --version
git --version

# Backend, from backend/
./mvnw spring-boot:run

# AI service, from ai-service/
python3 -m venv .venv
source .venv/bin/activate
pip install fastapi uvicorn
uvicorn app.main:app --reload --port 8000

# Frontend, from frontend/
npm install
```

```
npm run dev
```

Configure secrets through environment variables or local untracked configuration. Do not commit database passwords, JWT signing keys or private API keys.

11. Testing plan

Test area	Example cases
Authentication	Valid login, invalid password, expired token, disabled account.
Authorization	Citizen cannot access another citizen's private report; staff cannot update unassigned tasks.
Report creation	Missing description, invalid coordinates, unknown category, oversized media.
Workflow	Valid status transition, invalid transition, status-history entry created.
Assignment	Officer assigns within their department; unauthorized department assignment rejected.
AI service	Known keyword category, unknown category, empty/oversized input, service timeout.
Frontend	Form errors, upload progress, loading state, no reports state, mobile layout.

Acceptance criteria

• A citizen can create a report and receive a reference code.

• The citizen can see only their own reports unless another explicit role-based permission applies.

• An officer can assign a report to an eligible staff member.

• Every status update is timestamped and attributed to an authenticated user.

• Unauthorized users cannot view private report details or upload evidence to a report they do not control.

• AI suggestions are clearly labelled as suggestions and can be corrected by a human.

12. Privacy, security and accessibility

• Use BCrypt or Argon2 password hashing; never store plain-text passwords.

• Use short-lived signed access tokens and server-side role/ownership checks.

• Validate and scan uploaded images; enforce size limits and safe file handling.

• Do not expose exact citizen contact details or private metadata on public maps.

• Use HTTPS, secure secret management, database backups and audit logs.

• Provide accessible form labels, keyboard navigation, readable contrast and clear error messages.

• Set a retention policy for photos and personal information; collect only what is necessary.

13. Deployment plan

Docker Compose – local PostgreSQL

```
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_DB: civicfix
      POSTGRES_USER: civicfix_user
      POSTGRES_PASSWORD: local_dev_password
    ports:
      - "5432:5432"
    volumes:
      - civicfix_data:/var/lib/postgresql/data
volumes:
  civicfix_data:
```

• Build and test Spring Boot: ./mvnw clean test package.

• Build frontend: npm run build.

• Run PostgreSQL migrations using Flyway or Liquibase.

• Deploy backend and AI service with secrets stored in the host's secret manager.

• Deploy frontend to static hosting and configure the API URL.

• Enable HTTPS, restricted database access, logs, backups and health checks.

• Run smoke tests for citizen report, officer assignment and status tracking.

14. GitHub repository workflow

```
# Run from civicfix-ai project root
git init
git add .
git commit -m "Initial CivicFix AI project"
git branch -M main
git remote add origin https://github.com/YOUR-USERNAME/civicfix-ai.git
git push -u origin main
```

.gitignore essentials

```
# Java
target/
*.class

# Node
node_modules/
dist/

# Python
.venv/
__pycache__/
*.pyc

# Secrets and local files
.env
.env.*
!.env.example
.DS_Store
```

Add a README with project overview, architecture diagram, setup commands, environment variable names, API docs, test instructions and screenshots. Use synthetic demo reports and locations in public screenshots.

15. Project report structure

• Title page, certificate, declaration and acknowledgements (as required by college).

• Abstract, keywords and table of contents.

• Introduction, problem statement and objectives.

• Literature review / existing system and proposed system.

• Feasibility and requirement analysis.

• Architecture, use-case diagram, sequence diagrams and ER diagram.

• Database design, API design and implementation.

• AI methodology, dataset description and evaluation (if ML is implemented).

• Testing, results, screenshots and limitations.

• Conclusion, future scope and references.

16. Viva questions

Question	What to explain
Why PostgreSQL/PostGIS?	Relational constraints plus spatial indexing and geographic queries.
How prevent users seeing another person's report?	Authorization based on authenticated identity and ownership checks in the service layer.

Question	What to explain
Why keep status history?	Auditability, transparency and a reliable timeline of changes.
What does the AI model do?	Suggests a category; a human confirms it. Explain evaluation and limitations.
How handle duplicate reports?	Location/time/category similarity can flag candidates; do not auto-merge without review.
How scale the system?	Pagination, indexes, object storage, async media processing, caching and horizontal API scaling.
What is PostGIS?	A PostgreSQL extension that adds spatial data types, functions and indexes.

17. Suggested 8-week build schedule

Week	Milestone
1	Requirements, wireframes, ER diagram, repository and local environment.
2	Authentication, user roles, department and category setup.
3	Citizen report creation, media metadata and report tracking.
4	Officer dashboard, assignment and status workflow.
5	Map display, location filtering and admin analytics.
6	AI category suggestion baseline and evaluation.
7	Security checks, tests, accessibility and bug fixing.
8	Docker, deployment, screenshots, report, demo and viva preparation.

18. Final checklist

- All roles and workflows demonstrated with synthetic data.
- Backend authorization and ownership tests pass.
- Database migration scripts included.
- Frontend works at desktop and mobile widths.
- AI claims match what is actually implemented and evaluated.
- README reproduces setup from a clean clone.
- API documentation, screenshots and project report included.
- No passwords, keys or real citizen personal data committed.
- Student can explain the architecture, schema, security and limitations.

Portfolio tip: In interviews, demonstrate one end-to-end flow: citizen submits a report, officer assigns it, field staff updates progress, and citizen sees the status timeline. Explain the security checks and database transaction behind each step.

End of CivicFix AI project guide.